Python Primer - Experiment 0

Applied Machine Learning | Before Experiment 1

This single notebook covers every Python concept you need **before** starting the 12 PlacementPredict experiments. Each concept gets its own section with comments explaining *what* it is and *why* you'll need it. The last section stitches all 8 concepts together into one mini end-to-end example, using the same dataset you'll use for the real experiments.

Concepts covered: variables & types, arithmetic/comparison operators, strings & f-strings, lists & indexing, dictionaries, if/elif/else, for loops, functions, and importing libraries.

```
from google.colab import drive
drive.mount('/content/drive')
```

1. Variables & Types

A variable is just a name that points to a value. Python figures out the *type* automatically - you never declare it yourself.

```
# --- VARIABLES & TYPES ---

x = 5              # an integer (whole number)
y = 3.14           # a float (decimal number)
name = "CGPA"      # a string (text) - always in quotes
is_placed = True   # a boolean (True/False)

# type() tells you what kind of value a variable holds.
# You will use this constantly to sanity-check data you loaded from a CSV.
print(x, type(x))
print(y, type(y))
print(name, type(name))
print(is_placed, type(is_placed))
```

```
5 <class 'int'>
3.14 <class 'float'>
CGPA <class 'str'>
True <class 'bool'>
```

2. Arithmetic & Comparison Operators

Arithmetic operators do math. Comparison operators ask a true/false question - this is how you will filter rows of a dataset (e.g. "is this student's CGPA above 8?").

```
# --- ARITHMETIC OPERATORS ---
a = 7
b = 2
print("a + b =", a + b)     # addition
print("a - b =", a - b)     # subtraction
print("a * b =", a * b)     # multiplication
print("a / b =", a / b)     # division (always gives a float)
print("a ** 2 =", a ** 2)   # exponent (a squared) - used in Mean Squared Error

# --- COMPARISON OPERATORS ---
cgpa = 8.4
print("cgpa == 8.4 :", cgpa == 8.4)   # equality check (note: double ==, not =)
print("cgpa > 8    :", cgpa > 8)      # greater than
print("cgpa < 6    :", cgpa < 6)      # less than

# --- LOGICAL OPERATORS: and / or / not ---
attendance = 85
placed = (cgpa > 8) and (attendance > 80)   # both conditions must be true
print("placed :", placed)
```

```
a + b = 9
a - b = 5
a * b = 14
a / b = 3.5
a ** 2 = 49
cgpa == 8.4 : True
cgpa > 8    : True
cgpa < 6    : False
placed : True
```

3. Strings & f-strings

Every `print()` in every experiment uses an f-string to insert a variable's value into a sentence. Put `f` right before the opening quote, and wrap variables in `{ }`.

```
# --- STRINGS & F-STRINGS ---

model_name = "Logistic Regression"
accuracy = 0.87345

# Old, clunky way (avoid this):
print("Accuracy of " + model_name + " is " + str(accuracy))
```

```
# f-string way (use this):
print(f"Accuracy of {model_name} is {accuracy}")

# ':.3f' inside the braces rounds a float to 3 decimal places -
# you will use this in almost every experiment to print clean metrics.
print(f"Accuracy of {model_name} is {accuracy:.3f}")
```

```
Accuracy of Logistic Regression is 0.87345
Accuracy of Logistic Regression is 0.87345
Accuracy of Logistic Regression is 0.873
```

4. Lists & Indexing

A list is an ordered collection of values in square brackets. You will use lists constantly to hold column names ('features') before feeding them into a model.

```
# --- LISTS & INDEXING ---

features = ["CGPA", "AptitudeTestScore", "CodingTestScore", "MockInterviewScore

print(features)          # the whole list
print(features[0])       # FIRST item - Python counts from 0, not 1
print(features[1])       # SECOND item
print(features[-1])      # LAST item - negative index counts from the end
print(len(features))     # how many items are in the list

# Slicing: features[start:end] gives you a sub-list (end is NOT included)
print(features[0:2])     # first two items

# Adding to a list
features.append("Internships")
print(features)
```

```
['CGPA', 'AptitudeTestScore', 'CodingTestScore', 'MockInterviewScore']
CGPA
AptitudeTestScore
MockInterviewScore
4
['CGPA', 'AptitudeTestScore']
['CGPA', 'AptitudeTestScore', 'CodingTestScore', 'MockInterviewScore', 'Internsh
```

5. Dictionaries

A dictionary stores `key: value` pairs in curly braces. You'll use these whenever you compare several models side by side (Experiments 5 and 8).

```
# --- DICTIONARIES ---

# key = model name (string), value = the actual model object
from sklearn.linear_model import Ridge, Lasso, ElasticNet

models = {
    "Ridge": Ridge(alpha=1.0),
    "Lasso": Lasso(alpha=0.01),
    "ElasticNet": ElasticNet(alpha=0.01),
}

print(models)                  # the whole dictionary
print(models["Ridge"])         # access one value by its key

# Looping over a dictionary's keys and values together:
for name, model in models.items():
    print(f"Model name: {name}  ->  Object: {model}")
```

```
{'Ridge': Ridge(), 'Lasso': Lasso(alpha=0.01), 'ElasticNet': ElasticNet(alpha=0.
Ridge()
Model name: Ridge  ->  Object: Ridge()
Model name: Lasso  ->  Object: Lasso(alpha=0.01)
Model name: ElasticNet  ->  Object: ElasticNet(alpha=0.01)
```

6. if / elif / else

Conditional logic lets your code make decisions. You'll use this for things like flagging leakage, checking depth limits, or bucketing a score into a tier.

```
# --- IF / ELIF / ELSE ---

cgpa = 8.2

if cgpa >= 9:
    tier = "High"
elif cgpa >= 7:
    tier = "Mid"
else:
    tier = "Low"

print(f"CGPA {cgpa} falls into tier: {tier}")

# You can also use if/else to guard against bad data,
# e.g. before dividing or indexing:
missing_count = 0
if missing_count == 0:
    print("No missing values - safe to proceed.")
```

```
else:
    print("Missing values found - impute before modelling.")
```

```
CGPA 8.2 falls into tier: Mid
No missing values - safe to proceed.
```

7. for Loops

A for loop repeats a block of code once per item in a list. You'll use this for every hyperparameter sweep (tree depth, cluster count k, regularisation strength).

```
# --- FOR LOOPS ---

depths = [2, 4, 6, 8, 10]

for depth in depths:
    print(f"Training a tree with max_depth = {depth}")

# range() generates a sequence of numbers without writing them all out
for epoch in range(5):          # 0, 1, 2, 3, 4
    print(f"Epoch {epoch}: computing gradient...")

# Looping with an index AND a value using enumerate()
for i, d in enumerate(depths):
    print(f"Sweep step {i}: depth={d}")
```

```
Training a tree with max_depth = 2
Training a tree with max_depth = 4
Training a tree with max_depth = 6
Training a tree with max_depth = 8
Training a tree with max_depth = 10
Epoch 0: computing gradient...
Epoch 1: computing gradient...
Epoch 2: computing gradient...
Epoch 3: computing gradient...
Epoch 4: computing gradient...
Sweep step 0: depth=2
Sweep step 1: depth=4
Sweep step 2: depth=6
Sweep step 3: depth=8
Sweep step 4: depth=10
```

8. Functions (def / return)

A function packages a block of code under a name so you can reuse it. Nearly every experiment defines at least one function (e.g. `summarise_dataset`, `build_pipeline`).

```
# --- FUNCTIONS
```

```
# --- FUNCTIONS ---

def classify_cgpa(cgpa):
    """Takes a CGPA value and returns a tier label."""
    if cgpa >= 9:
        return "High"
    elif cgpa >= 7:
        return "Mid"
    else:
        return "Low"

# Calling the function with different inputs:
print(classify_cgpa(9.5))
print(classify_cgpa(7.8))
print(classify_cgpa(5.2))

# A function with more than one input (parameter):
def describe_model(name, accuracy):
    return f"{name} achieved {accuracy:.1%} accuracy"

print(describe_model("Random Forest", 0.913))
```

```
High
Mid
Low
Random Forest achieved 91.3% accuracy
```

9. Importing Libraries

`import` brings in code someone else wrote (pandas, scikit-learn, etc.) so you don't reinvent it. Every single experiment cell starts with imports like these.

```
# --- IMPORTING LIBRARIES ---

import pandas as pd                          # gives pandas the short name "pd"
import numpy as np                           # gives numpy the short name "np"
from sklearn.linear_model import LogisticRegression   # import ONE class from a
from sklearn.metrics import accuracy_score

# Colab-only: install a library that isn't there by default (note the leading !
# !pip install xgboost

print("pandas version:", pd.__version__)
print("numpy version :", np.__version__)
```

```
pandas version: 3.0.2
numpy version : 2.4.4
```

10. Putting It All Together - a Mini End-to-End Example

This cell uses **every concept above** on the real PlacementPredict dataset: variables/types, operators, f-strings, lists, dictionaries, if/elif/else, for loops, functions, and imports. Read the comments - each line is labelled with which concept it's using.

```
# 9. IMPORTING LIBRARIES
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression, Ridge
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# 1. VARIABLES & TYPES
csv_path = "placement_predict_50k_Dataset.csv"     # a string
min_acceptable_accuracy = 0.7                        # a float

# 4. LISTS
features = ["CGPA", "AptitudeTestScore", "CodingTestScore", "AttendancePer

df = pd.read_csv('/content/placement_predict_50k Dataset.csv')
df[features] = df[features].fillna(df[features].median())   # handle missi
X, y = df[features], df["PlacementStatus"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# 5. DICTIONARIES - comparing two models by name
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Decision Tree": DecisionTreeClassifier(max_depth=5, random_state=42),
}

# 8. FUNCTION definition
def evaluate_model(model, X_train, y_train, X_test, y_test):
    """Fits a model and returns its test accuracy."""
    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    return accuracy_score(y_test, preds)

# 7. FOR LOOP over the dictionary + 6. IF/ELIF/ELSE + 3. F-STRINGS
for name, model in models.items():
    acc = evaluate_model(model, X_train, y_train, X_test, y_test)   # 2. u

    if acc >= min_acceptable_accuracy:
        verdict = "GOOD"
    else:
        verdict = "NEEDS WORK"

    print(f"{name:<20} accuracy={acc:.3f}  -> {verdict}")
```

```
Logistic Regression  accuracy=0.877  -> GOOD
Decision Tree        accuracy=0.877  -> GOOD
```

You're ready for Experiment 1

Every line of code in the 12 PlacementPredict experiments is built from these 9 building blocks. When you hit something unfamiliar in a later experiment (e.g. `zip()`, `with ... as`, or a `class`), it will be introduced right before the experiment that first needs it - you don't need to learn it now.